* The Zero Value:

Go like most morden languages, assigns a default zero value to any variable that is declared but not assigned a value. having an explicit zero value makes code clearer and removes a source of bug.

* Literals.:

A Go literal is an explicitly specified number, character, or string. Go programs have four common kinds of literals.

- A rune literal represents a character and is surrounded by single qoutes. unlike many other languages. in Go single qoutes and double qoutes not interchangeable.

-: one rune literal backslash is not legal in a string literal: the single quote escape. It is replaced by a backslash escape for double quotes.

* Integer Types::

Type name	Value range
int8	–128 to 127
int16	–32768 to 32767
int32	–2147483648 to 2147483647
int64	–9223372036854775808 to 9223372036854775807
uint8	0 to 255
uint16	0 to 65535
uint32	0 to 4294967295
uint64	0 to 18446744073709551615

-: There are two other special names for integer types. rune and uintptr.

- The result of an integer division is an integer; if you want to get floating point result, you need to use type conversion to make your integers into floating-point numbers. Also, be careful not to divide an integer by 0; this causes panic.

- Integer division in Go follows truncation toward zero;

* Floating-point Types:

Type name	Largest absolute value	Smallest (nonzero) absolute value
`float32`	3.402823466385288598117041834845169254 40e+38	1.401298464324817070923729583289916131280e-45
`float64`	1.797693134862315708145274237317043567981e+308	4.940656458412465441765687928682213723651e-324

- Just like other languages, Go floating point numbers have a huge range, but they cannot store every value in that range; they store the nearest approximation. Because floats aren't

exact, they can be used only in situations where inexact value are acceptable or the rules of floating point are well understood.

- A floating-point number cannot represent a decimal value exactly. Do not use them to represent money or any other value that must have an exact decimal representation.

A Taste of Strings and Runes:

Strings in Go are immutable; you can reassign the value of a string variable, but you cannot change the value of the string that is assigned to it.

- Go also has a type that represents a single code point. The rune type an alias for int32 type, just us byte

is an alias for uint8.

+ Explicit Type Conversion -:

Go doesn't allow automatic type promotion between variables you must use a type conversion when variable types do not match. Even different-sized integers and floats must be converted to the same type to interact.

-q- Type conversions are one of the places where Go chooses to add a little verbosity in exchange for a great deal of simplicity and clarity. You'll see this trade-off multiple times. Idiomatic Go values comprehensibility over conciseness.

* Literals Are Untyped -:

Go lets you use an integer literal in floating-point expression

or even assign an integer literal in floating-point variable:

```
var x float64 = 10
var y float64 = 200.3 + 5
```

This is because, as I mentioned earlier, literals in Go are untyped.

* var versus :=

The := operator can do one trick that you cannot do with var: it allows you to assign values to existing too. As long as at least one new variable is on the lefthand side of the :=, any of the other variables can already exist:

```
x := 10
x, y := 30, "hello"
```

Using := has one limitation. If you are declaring a variable at the package level, you must use var because := is not legal outside of functions.

- Avoid declaring variables outside of functions because they complicate data flow analysis.

- Constants in Go are a way to give names to literals. There is no way in Go to declare that a variable is immutable.

* Typed and untyped constants:

constants can be typed or untyped. An untyped constant works exactly like a literal; it has no type of it's own but does have a default type that is used when no other type can be inferred. A typed constant can be directly assigned only to a variable of that type.

* Unused Constants:

perhaps surprisingly, the Go

Compiler allows you to create unread constants with const. This is because constants in Go are calculated at compile time and cannot have any side effects. This makes them easy to eliminate. if constant isn't used. it is simply not included in the compiled binary.